Backend-for-Frontend (BFF) Multi-Service Insurance Platform (Customer & Agent Portals) Architecture Design

(Submitted by: Leonard Wong Wei Lun)

Overview

This document provides an architecture designed of Backend-for-Frontend (BFF) to support two different frontend applications, instead of single monolithic design:

- Customer Portal (self-service policy purchase)
- Agent Portal (assisted purchase with advanced capabilities)

I. Assume Microservice Endpoints

To establish a baseline for the BFF's orchestration, we assume the downstream microservices expose the following capabilities:

- **Quotation Microservices**
  - POST /api/v1/quotes
    - Generate new quotation
  - GET /api/v1/quotes/{quoteId}
    - Retrieve quotation details
  - PUT /api/v1/quotes/{quoteId}
    - Update quotation details
  - GET /api/v1/customers/{customerId}/quotes
    - List of quotations from customers

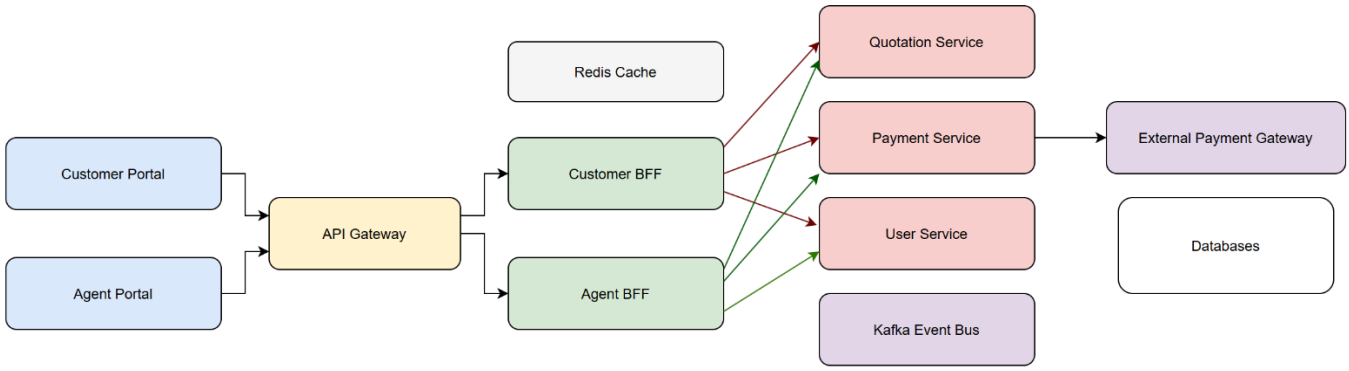
- **Payment Microservice (Idenmpotency support)**
  - POST /api/v1/payments
    - Initial payment
  - GET /api/v1/payments/{paymentId}
    - Get payment status
  - POST /api/v1/payments/{paymentId}/confirm
    - Confirm payment (webhook-driven)

- **User / Profile Service APIs**
  - GET /api/v1/customers/{customerId}
    - Get customer profile
  - POST /api/v1/customers
    - Create new customer
  - PUT /api/v1/customers/{customerId}
    - Update customer information
  - GET /api/v1/agents/{agentId}
    - Get agent profile
  - POST /api/v1/agents/{agentId}/customers/{customerId}
    - Assign customer to agent

- **External Payment Gateway**
  - POST /v1/payment_intents
    - Create payment session
  - POST /v1/payment_intents/{id}/confirm
    - Confirm payment session
  - GET /v1/payment_intents/{id}
    - Retrieve payment
  - POST /webhooks/payment
    - Secure external payment gateway webhooks

II. Architecture Diagram

Below is the overview of the architecture diagram



III. Solution Design

1. Architecture strategy: Dual BFF Pattern

Instead of a single monolithic front end, we can implement two different BFF services

Customer BFF: Optimize payload for lower bandwidth and abstract away aggregate Quotation and payment from client.

Agent BFF: Focus on complex data needs such as Quotation calculations, policy linking, commissions and allow agents to search for customer records.

2. Security, Authentication & Authorization

Given the sensitivity of personal data and secure payment, zero-trust approach is required

Authentication: Delegate to an external Identity Provider via OAuth 2.0 / OpenID Connect. Frontend receives short-lived JWTs (JSON Web Tokens).

Authorization: To validate the JWT signature and expiration and enforce Role-Based Access Control (RBAC) to ensure both agent and customer has permission to access specific module.

In-Transit Security: External traffic is secured via TLS 1.3. Internal traffic between the BFFs and microservices utilizes Mutual TLS (mTLS) to prevent internal spoofing.

Payment Security: Apply PCI-DSS compliant gateway so that system will not touch raw credit card numbers.

API Protection: Apply Rate Limiting & Web Application Firewall (WAF) to prevent Distributed Denial of Service (DDoS) attacks and improve system stability.

3. Data Consistency, Integrity & Payment Handling

Financial transactions require strict consistency guarantees, especially when distributed across two services (Payment & Quotation)

Idempotency: Frontends will generate a unique Idempotency-Key during initial payment, this is to ensure that payment microservice can you this to prevent customers double charge when encounter any issue during payment process.

Distributed Transaction (Saga Pattern): Implement an orchestrated Saga for the check out process.

1. BFF instructs the Quotation service to lock the quote
2. BFF calls the Payment Service to initial the charge
3. If payment succeeds, Payment Service will emit a Payment Completed event via message broker such as Kafka for Quotation service to consume and update the status
4. If payment fails, it will issue compensation transactions and unlock the quote.

4. Observability Strategy

To ensure the overall system health and stable, three pillars of Observability will be implemented.

Distributed Tracing (OpenTelemetry): A unique trace-id generated via API Gateway layer and injected into header of all downstream microservice calls. This allows engineers to visualize the entire lifespan of a request.

Metrics & Alerts (Grafana): Tracking the metrics such as Latency, Error Rate & TimeTaken for every endpoints and configure to send alerts when it hitting certain thresholds.

Structured Logging (Kibana): All services are output in JSON format logs and enable engineers for faster debugging, easier querying and root cause analysis.

5. Deployment Strategy

To minimize downtime and faster deployment, two strategies will be implemented

Containerization (Docker): BFFs and microservices will be packed as Docker Containers for deployment.

Auto Scaling (Kubenates): Kubenates enable auto deployment and Horizontal Pod Scaling (HPA) base on CPU utilization, to ensure system can handle during peak period.

IV. Conclusion

This architecture designed of Backend-for-Frontend (BFF) delivers a secure, scalable, and tailored integration layer that efficiently orchestrates microservices while providing optimized experiences for both customer and agent portals.